

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №8
по дисциплине «Нейронные сети»
Тема: Генерация текста на основе “Алисы в стране чудес”

Студент гр. 1306

Шаврин А.П.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы.

Рекуррентные нейронные сети также могут быть использованы в качестве генеративных моделей.

Это означает, что в дополнение к тому, что они используются для прогнозных моделей (создания прогнозов), они могут изучать последовательности проблемы, а затем генерировать совершенно новые вероятные последовательности для проблемной области.

Подобные генеративные модели полезны не только для изучения того, насколько хорошо модель выявила проблему, но и для того, чтобы узнать больше о самой проблемной области.

Задание.

- Ознакомиться с генерацией текста
- Ознакомиться с системой Callback в Keras

Выполнение работы.

Данные и предобработка

Данные были взяты из проекта Гутенберг - книга Льюиса Кэрролла «Приключения Алисы в Стране чудес» в формате ASCII (UTF-8). Исходный текст содержал служебные заголовки и колонтитулы, которые были удалены. Для этого были найдены строки-маркеры начала и конца основного текста (рисунок 1).

Загрузка данных

```
def load_data(filename: str = "wonderland.txt"):
    with open(filename, 'r', encoding='utf-8') as f:
        raw_text = f.read()
    return raw_text
```

```
data_path = os.path.join(base_path, "wonderland.txt")
raw_text = load_data(data_path)
```

Предобработка данных

```
def remove_header_and_footer(text: str, header: str, footer: str) -> str:
    start_idx = max(0, text.find(header) + len(header))
    end_idx = min(len(text), text.find(footer))
    text = text[start_idx : end_idx].strip()
    return text
```

```
▶ # Приводим к нижнему регистру
raw_text = raw_text.lower()
header = "*** START OF THE PROJECT GUTENBERG EBOOK ALICE'S ADVENTURES IN WONDERLAND ***".lower()
footer = "*** END OF THE PROJECT GUTENBERG EBOOK ALICE'S ADVENTURES IN WONDERLAND ***".lower()

# Удаляем хедер и футер
print(f"Длина текста до очистки: {len(raw_text)} символов")
raw_text = remove_header_and_footer(raw_text, header, footer)

print(f"Длина текста после очистки: {len(raw_text)} символов")
print(f"Первые 500 символов:\n{raw_text[:500]}")
```

```
... Длина текста до очистки: 163951 символов
Длина текста после очистки: 144599 символов
Первые 500 символов:
[illustration]
```

```
alice's adventures in wonderland

by lewis carroll

the millennium fulcrum edition 3.0

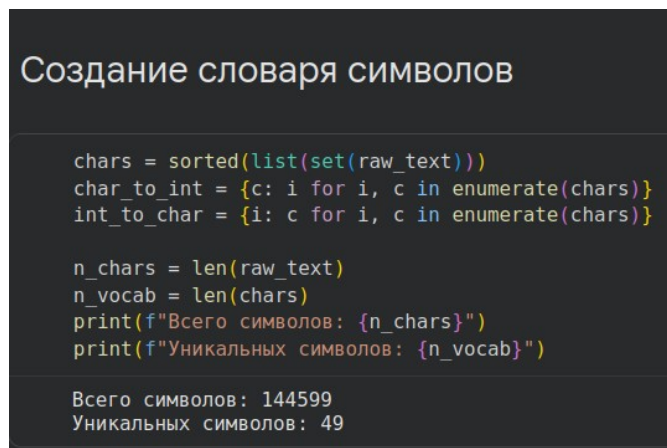
contents

chapter i.    down the rabbit-hole
chapter ii.   the pool of tears
```

Рисунок 1. Загрузка и очистка данных.

После очистки длина текста сократилась с 163 951 до 144 599 символов. Весь текст был приведен к нижнему регистру для уменьшения словаря. Приведение к нижнему регистру позволяет сети изучать символы без учета регистра, что уменьшает количество уникальных символов и упрощает обучение. Удаление служебных частей избавляет от не относящегося к книге шума.

Был создан список всех уникальных символов, встречающихся в тексте, и отображения `char_to_int` и `int_to_char`. Количество уникальных символов составило 49 (включая буквы, знаки препинания, пробелы и переводы строк). Это значение было взято как размер словаря `n_vocab` (рисунок 2).



```
Создание словаря символов

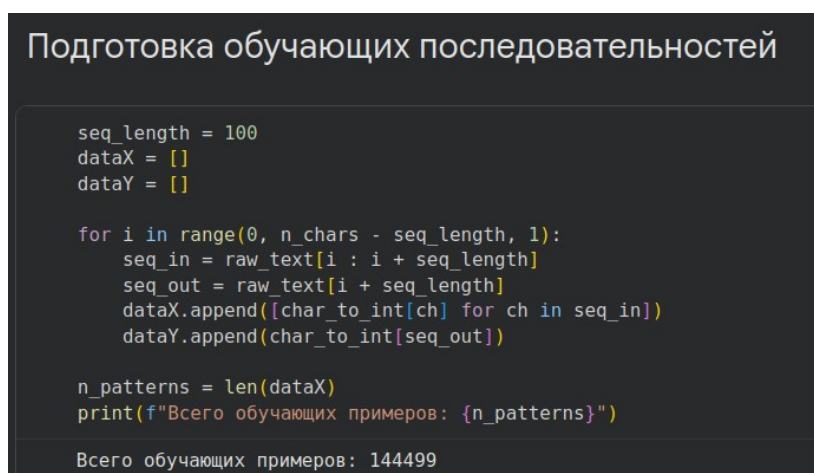
chars = sorted(list(set(raw_text)))
char_to_int = {c: i for i, c in enumerate(chars)}
int_to_char = {i: c for i, c in enumerate(chars)}

n_chars = len(raw_text)
n_vocab = len(chars)
print(f"Всего символов: {n_chars}")
print(f"Уникальных символов: {n_vocab}")

Всего символов: 144599
Уникальных символов: 49
```

Рисунок 2. Создание словаря символов.

Для обучения сети были созданы пары вход - выход: входная последовательность длиной 100 символов и следующий за ней один символ (целевой). Окно сдвигалось по тексту с шагом 1. Таким образом было сформировано 144 499 обучающих примеров. Длина 100 символов позволяет сети улавливать долгосрочные зависимости, например, грамматические конструкции, повторяющиеся фразы, стилистические особенности. Выбор этого значения является компромиссом между вычислительной сложностью и качеством захвата контекста (рисунок 3).



```
Подготовка обучающих последовательностей

seq_length = 100
dataX = []
dataY = []

for i in range(0, n_chars - seq_length, 1):
    seq_in = raw_text[i : i + seq_length]
    seq_out = raw_text[i + seq_length]
    dataX.append([char_to_int[ch] for ch in seq_in])
    dataY.append(char_to_int[seq_out])

n_patterns = len(dataX)
print(f"Всего обучающих примеров: {n_patterns}")

Всего обучающих примеров: 144499
```

Рисунок 3. Подготовка обучающих последовательностей.

Входные данные X были преобразованы в трехмерный тензор формы (образцы, временные шаги, признаки) = (144499, 100, 1). Выполнена нормализация значений (деление на `n_vocab`) для приведения к диапазону [0,1], что способствует стабильности градиентов в LSTM. Целевые значения y преобразованы в one-hot векторы размера `n_vocab`. Это позволяет сети решать задачу классификации на 49 классов. Слой LSTM (Long Short-Term Memory) был выбран, поскольку он специально предназначен для работы с последовательностями, запоминая долгосрочные зависимости и справляясь с проблемой затухающих градиентов для генерации текста на уровне символов.

Затем была построена модель Sequential:

- Слой LSTM с 256 нейронами. Возвращаемое значение — последний выходной вектор (по умолчанию).
- Слой Dropout с вероятностью 0.2 для регуляризации и предотвращения переобучения.
- Полносвязный слой Dense с softmax-активацией, выдающий вероятности для каждого из 49 символов.

Колбеки для управления обучением

Был реализован 1 самописный колбек и настроены 2 колбека предоставляемых Keras (рисунок 4)

1. ModelCheckpoint - Сохранял веса модели после каждой эпохи, если значение потерь улучшалось. Имя файла включало номер эпохи и значение потерь. Это позволило в дальнейшем загрузить лучшие веса для генерации.

2. TensorBoard - Колбек TensorBoard записывал логи обучения в директорию logs/fit/.

3. Самописный колбек TextGeneratorCallback — был реализован для контроля качества генерируемого текста во время обучения. Он каждые `every_n_epochs` эпох выбирал случайный seed (последовательность из 100 символов из обучающих данных) и генерировал 200 символов с помощью

текущей модели. Вывод печатался в консоль. Это позволило оценить, как улучшается качество генерации по мере снижения потерь.

Callback для генерации текста

```
class TextGeneratorCallback(Callback):
    def __init__(self, dataX, int_to_char, vocab_size, generate_length=200, every_n_epochs=5):
        super().__init__()
        self.dataX = dataX
        self.int_to_char = int_to_char
        self.vocab_size = vocab_size
        self.generate_length = generate_length
        self.every_n_epochs = every_n_epochs

    def on_epoch_end(self, epoch, logs=None):
        if (epoch + 1) % self.every_n_epochs != 0:
            return

        print(f"\n\n--- Генерация текста после эпохи {epoch+1} ---")
        self.generate_text()
        print("\n--- Конец генерации ---\n")

    def generate_text(self):
        start = np.random.randint(0, len(self.dataX) - 1)
        pattern = self.dataX[start].copy()

        print("Seed:")
        print(''.join([self.int_to_char[value] for value in pattern]))

        generated = generate_text(
            self.model,
            pattern,
            self.int_to_char,
            self.vocab_size,
            self.generate_length
        )

        print("\nGenerated text:")
        print(generated)
```

Callbacks

```
checkpoint_path = os.path.join(base_path, "weights-improvement-{epoch:02d}-{loss:.4f}.keras")
checkpoint = ModelCheckpoint(
    filepath=checkpoint_path,
    monitor='loss',
    verbose=1,
    save_best_only=True,
    mode='min'
)

log_dir = os.path.join(base_path, "logs/fit/" + datetime.datetime.now().strftime("%Y%m%d-%H%M%S"))
tensorboard_callback = TensorBoard(
    log_dir=log_dir,
    histogram_freq=1,
    write_graph=True
)

text_gen_callback = TextGeneratorCallback(
    dataX=dataX,
    int_to_char=int_to_char,
    vocab_size=n_vocab,
    generate_length=200,
    every_n_epochs=5
)

callbacks_list = [checkpoint, tensorboard_callback, text_gen_callback]
```

Рисунок 4. Колбеки.

Обучение модели

Обучение проводилось в два этапа:

- Этап 1: 20 эпох, размер батча 128. Значение потерь (categorical crossentropy) снизилось с 3.0087 до 1.9971.

- Этап 2: дообучение с 20-й по 30-ю эпоху (10 эпох) с загрузкой последних весов. Потери продолжили снижаться до 1.8073.

Динамика потерь демонстрирует устойчивое улучшение на протяжении всего обучения, что говорит о хорошей сходимости. В ходе обучения производился вывод генераций текста каждые 5 эпох, примеры seed значений, сгенерированного текста и объяснения результата приведены в таблице 1.

Таблица 1. Генерации в ходе обучения.

Эпоха	Seed	Результат генерации	Анализ
5	piece if you want to see how he did it,) he did not look at all comfortable, and it was certainly no	the tooee th the tas nhe toe toe toee the woued toe toee to the woel she woued toe toee to the woel she was nhe toured to the woel she woued toe toee to the woee she was nhe toe toe toee to th	На этом этапе модель еще не выучила осмысленных слов. Текст состоит из повторяющихся слогов и часто встречающихся сочетаний (the, to, toe, woel, she. Это ожидаемо, так как сеть только начинает улавливать статистические закономерности.
10	you're doing!" cried alice, jumping up and down in an agony of terror. "oh, there goes his _precious	"hu a a dont the woudt tou do aelut the woudt toued to toe toee and the was toen i sas an the was oo tie taate and the was toen i sas anl the was oo tie taate and the was toen i sas anl the wa	Появляются отдельные короткие слова (dont, the, and), но в целом текст остается бессмысленным. Модель начала улавливать, что после пробелов часто идут определенные символы, но грамматика отсутствует.
15	fact."i keep them to sell," the hatter added as an	r io wou den the was to the thing satee she was toen i sood fr the wiile tas soen i fen	Появляются фрагменты слов, напоминающие настоящие (rabbit, thing). Модель начинает

Эпоха	Seed	Результат генерации	Анализ
	explanation; “i’ve none of my own. i’m a hatte	to the then and the was the rabbit sas sht oo har and the thie the was she rabbit sas sht inr to the that sh	генерировать повторяющиеся структуры с чередованием she was, the rabbit, что указывает на выучивание некоторых имен и глаголов.
20	her’s latin grammar, “a mouse—of a mouse—to a mouse—a mouse—o mouse!”) the mouse looked at her rathe	r “it was the maat tf the cace, “h wonle ”ou a	Генерация стала более структурированной, но все еще далека от связного текста. Появились знаки препинания и переводы строк, что говорит о выучивании форматирования.
25	at the hatter, and, just as the dormouse crossed the court, she said to one of the officers of the	cour, and whin samk to tee white rasee and tooe oi the tood. “ho wou del iot to the moos,” said the hatter. “i sontt shet youl toe sooeo oo the soos.” the mabte was aelin an in an angrue of thoe sh	Здесь уже видны попытки построить диалоги (said the hatter), появляются кавычки, знаки препинания. Некоторые слова (white, «mouse») угадываются, хотя и с ошибками. Текст начинает напоминать по структуре книжный стиль.

Эпоха	Seed	Результат генерации	Анализ
30	finish my tea,” said the hatter, with an anxious look at the queen, who was reading the list of sing	t she would balk thi hor hnr herd and the could and whnn she sas oo tie lirtle doos, and the suoen tu ten a loeat furt and whin she was gow the was oo tirting on the sood. “ho y said the docm sa	Генерация стала более разнообразной, появляются новые персонажи (the queen, the hatter), но все еще много ошибок в написании. Модель научилась имитировать диалоги и повествование, но не выучила корректное написание многих слов.

На 20й эпохе можно заметить, что после последнего символа а идут длинные пробелы, что означает, что модель на большом количестве шагов генерировала пробелы (или символы, которые не выводятся в консоль как печатные). В коде генерация всегда идет 200 шагов (указано `generate_length=200` в колбэке), но в выводе эти 200 символов превратились в визуально короткую строку из-за того, что большинство символов - это пробелы или символы новой строки. Данный эффект мог произойти по ряду причин:

- Метод `np.argmax(prediction)` выбирает символ с наибольшей предсказанной вероятностью. На ранних этапах обучения распределение вероятностей часто бывает плоским (много символов с близкой вероятностью)
- В английском тексте пробел - один из самых частых символов. На 20-й эпохе `loss` еще достаточно высок (~ 2.0), и модель еще не выучила сложные зависимости, поэтому ее предсказания могут быть смещены в сторону самых частых символов.

- Выбранный seed (начальная последовательность) может повлиять на то, в какую сторону пойдет генерация. Если seed содержит редкие символы или нестандартную комбинацию, модель может на ранних шагах сгенерировать пробел и затем «застрясть» в нем, так как argmax будет повторять тот же символ.

Финальная генерация с лучшими весами

После завершения обучения были загружены веса с наименьшими потерями (weights-improvement-30-1.8073.keras) и сгенерирован текст длиной 500 символов.

Seed:

picked up.”

“what’s in it?” said the queen.

“i haven’t opened it yet,” said the white rabbit, “bu

Сгенерированный текст (фрагмент):

t it you werl toe tao of to head to tey at io io’ io dian!” she said to herself, “ih moek
toit a san ‘hat io the sooeo of the raabit tole! and the whit hir tha sao of the tabeit tay
ani the wind was oh the rirel oaad tote and saed to the coortes

and whnt simnt sas ani the winte to her and sooe of the was of the riaee si thr what
the was no aalit and toone and tooei and the whole to her ont and the whnd was oh
the court,

“i whol t aad davery yeut in an ince ai i fenn ”

thought alic. “i” a

Финальный текст показывает, что модель научилась генерировать последовательности, напоминающие английский язык, но все еще страдает от галлюцинаций в виде несуществующих слов и неправильной грамматики. Это типично для генерации на уровне символов.

Анализ с помощью TensorBoard

TensorBoard предоставил возможность визуализировать процесс обучения.

График потерь (loss) на рисунке 5 показывает плавное снижение loss от ~3.0 до ~1.8. Отсутствие резких скачков и колебаний говорит о стабильности обучения.

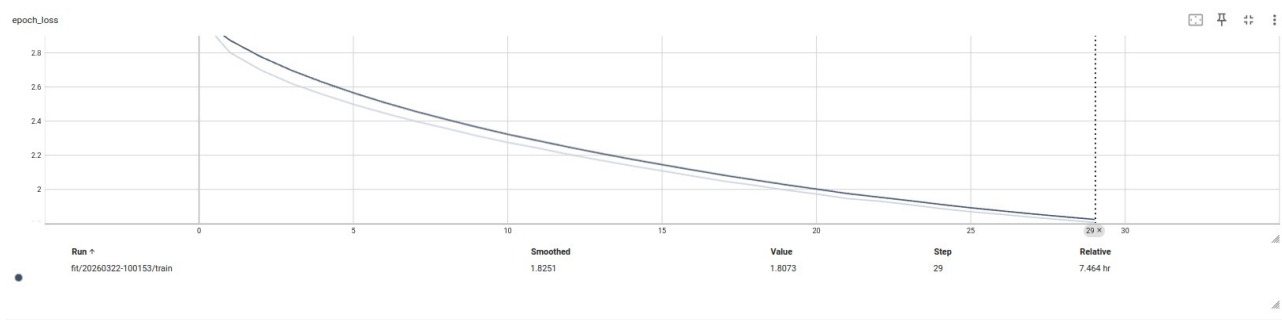


Рисунок 5. График потерь в ходе обучения.

Если посмотреть на динамику bias-гистограмм по эпохам (рисунок 6), то можно видеть:

- При инициализации bias обычно устанавливаются в ноль (или очень близко к нулю). Поэтому гистограмма в начале имеет один высокий пик около 0. Это означает, что все нейроны имеют практически одинаковые смещения, и сеть еще не начала настраиваться.
- К концу обучения оно становится более низким и широким. Вместо одного пика у нуля теперь несколько четких пиков в разных точках.

В процессе обучения разные нейроны LSTM адаптируются к различным паттернам в тексте. Одни нейроны отвечают за определенные символы или грамматические конструкции, и их bias подстраиваются под нужный уровень активации. В результате распределение bias становится многомодальным (несколько пиков), что свидетельствует о сформировавшейся специализации. Отрицательные bias заставляют нейрон активироваться только при достаточно сильном входном сигнале. Это помогает сети игнорировать шум и

фокусироваться на значимых закономерностях. Смещение в отрицательную область – признак того, что сеть научилась быть избирательной.

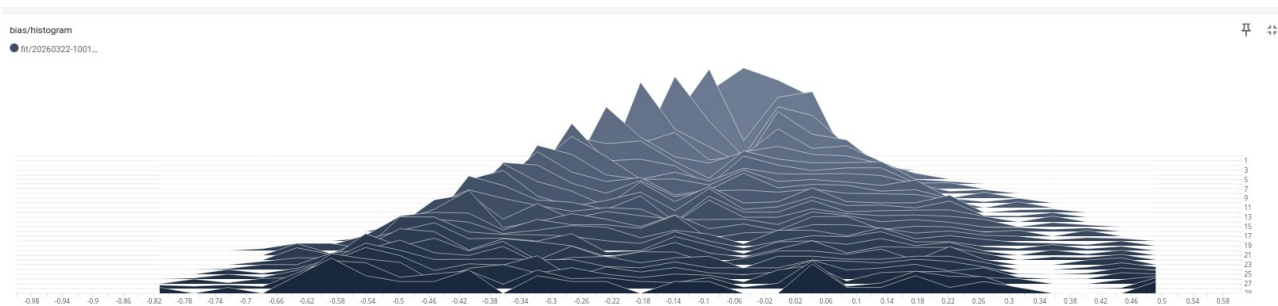


Рисунок 6. Гистограммы смещения в ходе обучения.

Гистограмма kernel/histogram показывает распределение весов (weights) на рисунке 7. При инициализации веса задаются маленькими случайными числами. Это приводит к тому, что большинство весов сосредоточено около нуля, а распределение имеет острый пик. В процессе обучения веса адаптируются под данные. Они перестают быть одинаково маленькими и начинают принимать более разнообразные значения, необходимые для кодирования различных паттернов текста. Сеть настраивает веса, чтобы научиться распознавать зависимости.

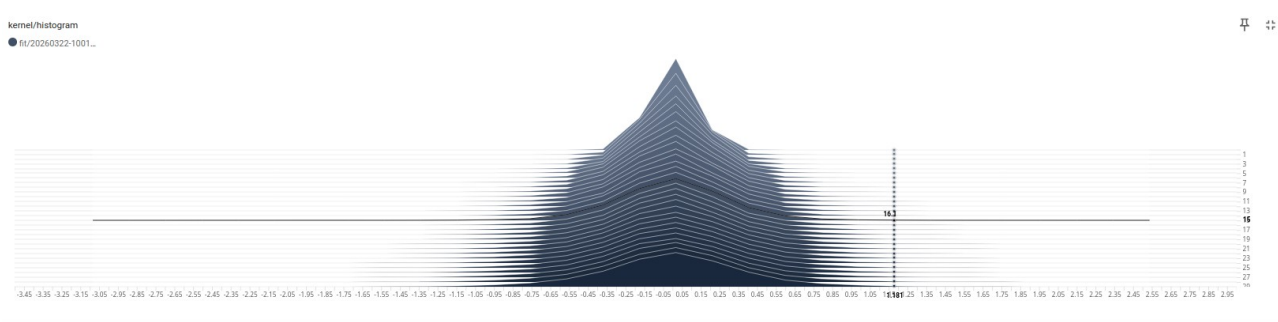


Рисунок 7. Гистограммы весов в ходе обучения.

Выводы.

В ходе работы была построена и обучена рекуррентная нейронная сеть LSTM для генерации текста на уровне символов. Модель на основе 30 эпох обучения смогла выучить статистические закономерности текста и генерировать последовательности, отдаленно напоминающие английский язык.

Самописный колбэк `TextGeneratorCallback` позволил контролировать качество генерации в процессе обучения. Наблюдение за генерируемыми текстами после каждой 5-й эпохи показало, как модель постепенно начинает улавливать структуру предложений, знаки препинания и отдельные слова.

Использование `TensorBoard` предоставило визуальную информацию о сходимости модели и распределении весов, что помогает в диагностике обучения.

Несмотря на относительно низкие потери (1.8), генерируемый текст все еще содержит множество орфографических ошибок и не является связным. Это объясняется тем, что символьный уровень не позволяет модели в полной мере выучить семантику. Для более качественной генерации необходимо увеличивать глубину сети, использовать больше эпох или применять более сложные архитектуры.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: lab8.ipynb

```
"""# Установка зависимостей
"""
```

```
!pip install pandas numpy matplotlib scikit-learn tensorflow
```

```
"""# Импорт"""
```

```
import os
import numpy as np
import matplotlib.pyplot as plt
from keras.models import Sequential
from keras.layers import LSTM, Dense, Dropout
from keras.callbacks import ModelCheckpoint, Callback, TensorBoard
from tensorflow.keras.utils import to_categorical
import glob
import datetime
```

```
from google.colab import drive
drive.mount('/content/drive', force_remount=True)
```

```
base_path = "/content/drive/MyDrive/Colab Notebooks/NeuralNetworks/lab8"
```

```
"""# Загрузка данных"""
```

```
def load_data(filename: str = "wonderland.txt"):
    with open(filename, 'r', encoding='utf-8') as f:
        raw_text = f.read()
    return raw_text
```

```
data_path = os.path.join(base_path, "wonderland.txt")
raw_text = load_data(data_path)
```

```
"""# Предобработка данных"""
```

```
def remove_header_and_footer(text: str, header: str, footer: str) -> str:
    start_idx = max(0, text.find(header) + len(header))
    end_idx = min(len(text), text.find(footer))
    text = text[start_idx : end_idx].strip()
    return text
```

```
# Приводим к нижнему регистру
raw_text = raw_text.lower()
```

```
header = "**** START OF THE PROJECT GUTENBERG EBOOK ALICE'S  
ADVENTURES IN WONDERLAND ****".lower()
```

```
footer = "**** END OF THE PROJECT GUTENBERG EBOOK ALICE'S ADVENTURES  
IN WONDERLAND ****".lower()
```

```
# Удаляем хедер и футер
```

```

print(f"Длина текста до очистки: {len(raw_text)} символов")
raw_text = remove_header_and_footer(raw_text, header, footer)

print(f"Длина текста после очистки: {len(raw_text)} символов")
print(f"Первые 500 символов:\n{raw_text[:500]}")

"""# Создание словаря символов"""

chars = sorted(list(set(raw_text)))
char_to_int = {c: i for i, c in enumerate(chars)}
int_to_char = {i: c for i, c in enumerate(chars)}

n_chars = len(raw_text)
n_vocab = len(chars)
print(f"Всего символов: {n_chars}")
print(f"Уникальных символов: {n_vocab}")

"""# Подготовка обучающих последовательностей"""

seq_length = 100
dataX = []
dataY = []

for i in range(0, n_chars - seq_length, 1):
    seq_in = raw_text[i : i + seq_length]
    seq_out = raw_text[i + seq_length]
    dataX.append([char_to_int[ch] for ch in seq_in])
    dataY.append(char_to_int[seq_out])

n_patterns = len(dataX)
print(f"Всего обучающих примеров: {n_patterns}")

"""# Преобразование для Keras"""

X = np.reshape(dataX, (n_patterns, seq_length, 1))
# нормализация
X = X / float(n_vocab)
# one-hot кодирование
y = to_categorical(dataY)

print(f"Форма X: {X.shape}")
print(f"Форма y: {y.shape}")

"""# Создание модели"""

def build_model():
    model = Sequential()
    model.add(LSTM(256, input_shape=(X.shape[1], X.shape[2])))
    model.add(Dropout(0.2))
    model.add(Dense(y.shape[1], activation='softmax'))
    model.compile(loss='categorical_crossentropy', optimizer='adam')
    model.summary()

```

```

return model

"""# Генерация текста"""

def generate_text(model, pattern, int_to_char, vocab_size, length=200):
    pattern = pattern.copy()
    generated_text = ""

    for _ in range(length):
        X = np.reshape(pattern, (1, len(pattern), 1))
        X = X / float(vocab_size)

        prediction = model.predict(X, verbose=0)
        index = np.argmax(prediction)

        result = int_to_char[index]
        generated_text += result

        pattern.append(index)
        pattern = pattern[1:]

    return generated_text

"""# Callback для генерации текста"""

class TextGeneratorCallback(Callback):
    def __init__(self, dataX, int_to_char, vocab_size, generate_length=200,
every_n_epochs=5):
        super().__init__()
        self.dataX = dataX
        self.int_to_char = int_to_char
        self.vocab_size = vocab_size
        self.generate_length = generate_length
        self.every_n_epochs = every_n_epochs

    def on_epoch_end(self, epoch, logs=None):
        if (epoch + 1) % self.every_n_epochs != 0:
            return

        print(f"\n\n--- Генерация текста после эпохи {epoch+1} ---")
        self.generate_text()
        print("\n--- Конец генерации ---\n")

    def generate_text(self):
        start = np.random.randint(0, len(self.dataX) - 1)
        pattern = self.dataX[start].copy()

        print("Seed:")
        print(''.join([self.int_to_char[value] for value in pattern]))

        generated = generate_text(
            self.model,

```



```

        pattern,
        self.int_to_char,
        self.vocab_size,
        self.generate_length
    )

    print("\nGenerated text:")
    print(generated)

"""# Callbacks"""

checkpoint_path = os.path.join(base_path, "weights-improvement-{epoch:02d}-{loss:.4f}.keras")
checkpoint = ModelCheckpoint(
    filepath=checkpoint_path,
    monitor='loss',
    verbose=1,
    save_best_only=True,
    mode='min'
)

log_dir = os.path.join(base_path, "logs/fit/" + datetime.datetime.now().strftime("%Y%m%d-%H%M%S"))
tensorboard_callback = TensorBoard(
    log_dir=log_dir,
    histogram_freq=1,
    write_graph=True
)

text_gen_callback = TextGeneratorCallback(
    dataX=dataX,
    int_to_char=int_to_char,
    vocab_size=n_vocab,
    generate_length=200,
    every_n_epochs=5
)

callbacks_list = [checkpoint, tensorboard_callback, text_gen_callback]

"""# Обучение"""

def plot_history(history, title="Графики обучения"):
    plt.figure(figsize=(10, 5))
    plt.plot(history.history['loss'], label='Потери на обучении')
    plt.title(f'{title} - Потери')
    plt.xlabel('Эпоха')
    plt.ylabel('Loss')
    plt.legend()
    plt.grid(True)
    plt.show()

```

```

model = build_model()
history = model.fit(X, y, epochs=20, batch_size=128, callbacks=callbacks_list, verbose=1)

plot_history(history)

# Commented out IPython magic to ensure Python compatibility.
# %load_ext tensorboard
# %tensorboard --logdir "/content/drive/MyDrive/Colab
Notebooks/NeuralNetworks/lab8/logs"

start_epoch = 20
additional_epochs = 10
model2 = build_model()

weights_files = glob.glob(os.path.join(base_path, "weights-improvement-*.keras"))

if weights_files:
    weights_files.sort(key=lambda x: int(os.path.basename(x).split('-')[2]))
    last_weights = weights_files[-1]
    print(f"Загрузка весов из последнего файла: {last_weights}")
    model2.load_weights(last_weights)

    start_epoch = int(os.path.basename(last_weights).split('-')[2])
else:
    print("Файлы весов не найдены, используем текущие веса модели.")
    start_epoch = 0

model2.fit(
    X, y,
    epochs=start_epoch + additional_epochs,
    initial_epoch=start_epoch,
    batch_size=128,
    callbacks=callbacks_list,
    verbose=1
)

# Commented out IPython magic to ensure Python compatibility.
# %load_ext tensorboard
# %tensorboard --logdir "/content/drive/MyDrive/Colab
Notebooks/NeuralNetworks/lab8/logs"

"""# Загрузка лучших весов и финальная генерация"""

final_model = build_model()
weights_files = glob.glob(os.path.join(base_path, "weights-improvement-*.keras"))
if weights_files:
    best_weights = min(weights_files, key=lambda x: float(x.split('-')[3][:-6]))
    print(f"Загрузка лучших весов из файла: {best_weights}")
    if os.path.exists(best_weights):
        final_model.load_weights(best_weights)

```

```

        else:
            print(f"Загрузка лучших весов из файла: {best_weights} не удалась, файл не
найден")
        else:
            print("Не найдены файлы весов, используем текущие веса модели.")

start = np.random.randint(0, len(dataX) - 1)
pattern = dataX[start]

print("Seed:")
print(".".join([int_to_char[value] for value in pattern]))

print("\n--- Финальная генерация ---\n")

final_generation = generate_text(
    final_model,
    pattern,
    int_to_char,
    n_vocab,
    length=500
)

print(final_generation)

print("\n--- Конец ---")

```